EVIDENCIA DE PRUEBAS AUTOMÁTICAS

Proyecto Entre Platos y Copas

APT122 · Capstone

Fase	Fase 2
Integrantes	Helen Cortez y Benjamín Romero
Sección	002D
Resultado	160 pruebas · 159 aprobadas · 1 omitida · 0 fallidas
Cobertura	86 % del código de la aplicación

1. Objetivo

El objetivo de las pruebas automáticas es verificar que las principales reglas de negocio del sistema Entre Platos y Copas funcionen correctamente, incluyendo la creación de cotizaciones, confirmación de reservas, control de disponibilidad, preparación, despacho, devolución, registro de mermas y actualización del inventario.

A diferencia de la evidencia funcional (01 Aplicación), que muestra el sistema en uso, esta evidencia demuestra mediante código ejecutable que esas reglas se verifican de forma automática y reproducible.

Resumen de resultados

Total	Aprobadas	Omitidas	Fallidas	Tiempo	Cobertura
160	**159**	1	0	47,801 s	**86 %**

2. Entorno de pruebas

Lenguaje	Python 3.12.10
Framework	Django 6.1.1 (versiones de requirements.txt del proyecto)
Framework de pruebas	django.test.TestCase (unittest), con subTest para casos múltiples y TransactionTestCase para concurrencia
Base de datos	La configuración utiliza SQLite exclusivamente para pruebas. Aunque settings_test.py define una ruta SQLite local, durante la ejecución Django crea una base de datos de prueba temporal en memoria, identificada como memorydb_default, y la elimina al finalizar. No se usa la base de producción de Supabase.
Configuración	Epyc/settings_test.py (configuración de pruebas del propio proyecto)
Sistema operativo	Windows 11 (10.0.26200)
Archivo de pruebas	Inventario/tests.py · 2.090 líneas · 26 clases definidas: 25 grupos de pruebas y 1 clase base auxiliar (EscenarioReserva) · 160 pruebas
Código evaluado	Repositorio desplegado en Render (Benjamin-Romero-Cortes/Epyc), commit 457425e: https://github.com/Benjamin-Romero-Cortes/Epyc/commit/457425e
Cobertura	coverage 7.6.10
Fecha de ejecución	05/11/2026

Comandos utilizados

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt coverage
python manage.py test --settings=Epyc.settings_test --verbosity 2

coverage run --source=Inventario,gestion --omit="*/migrations/*,*/tests.py,*/templates/*" manage.py test --
settings=Epyc.settings_test
coverage report -m
coverage html
```

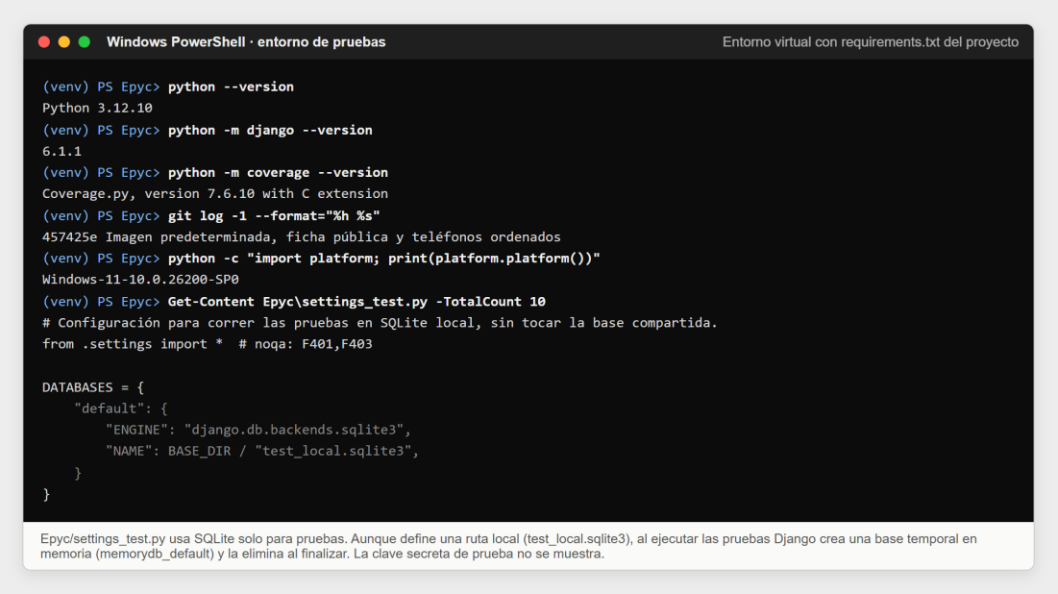


Figura 1. Entorno de pruebas (01_Entorno_Pruebas.png)

3. Matriz de pruebas

Pruebas representativas por módulo. Cada fila corresponde a métodos que existen en Inventario/tests.py y que se ejecutaron en la corrida documentada; el resultado se toma de la salida real. La lista completa de las 160 pruebas está en el Anexo.

ID	Módulo	Prueba (clase y método)	Resultado esperado	Resultado
PA-01	Clientes	ValidacionesTests test_normaliza_rut test_digito_verificador	El RUT se normaliza (12.345.678-k → 12345678K) y solo se acepta con dígito verificador válido.	**Aprobada**
PA-02	Clientes	CotizadorPublicoTests test_solicitud_valida_crea_cliente_y_servicio	Una solicitud web válida crea el cliente y un servicio en estado Cotización con sus categorías, sin artículos inventados.	**Aprobada**
PA-03	Clientes	CotizadorPublicoTests test_errores_de_validacion	RUT, fecha pasada, invitados, categorías, correo, teléfono, tipo de evento o nombre inválidos se rechazan (400) sin crear registros.	**Aprobada**
PA-04	Cotización	AdministracionTests test_catalogo_agrega_varios_productos_de_una_vez	Se agregan varios productos de una vez y el total queda en 45 × $500 + 20 × $300.	**Aprobada**
PA-05	Cotización	AdministracionTests test_catalogo_rechaza_cantidades_invalidas_sin_agregar_nada	Cantidades negativas, decimales o texto se rechazan y no se agrega ningún producto.	**Aprobada**
PA-06	Reserva	DisponibilidadTests test_1_sin_reservas_superpuestas	Con stock suficiente, la cotización pasa a estado Confirmado.	**Aprobada**
PA-07	Inventario	DisponibilidadTests test_2_reserva_existente_con_stock_suficiente	Dos reservas de 30 y 70 unidades comprometen 100 de 100: disponible 0.	**Aprobada**
PA-08	Inventario	DisponibilidadTests test_3_la_suma_supera_el_stock test_7_dos_confirmaciones_seguidas_no_sobrevenden	Si no alcanza el stock se rechaza la confirmación (solicitado 40, disponible 30, faltante 10) y no hay sobreventa.	**Aprobada**
PA-09	Inventario	DisponibilidadTests test_5_cotizacion_no_confirmada_no_bloquea test_4_reserva_en_otras_fechas_no_afecta	Cotizaciones sin confirmar, reservas canceladas y reservas en otras fechas no retienen stock.	**Aprobada**
PA-10	Inventario	CifrasUnificadasTests test_resumen_inventario_y_reportes_coinciden	Resumen, Inventario y Reportes muestran las mismas cifras de stock (70 disponibles de 130) y de alertas.	**Aprobada**
PA-11	Preparación	PreparacionTests test_8_y_9_flujo_completo_cambia_estados_y_deja_historial	Confirmado → Preparacion → Preparado, con responsable, fecha de finalización e historial de cada acción.	**Aprobada**
PA-12	Preparación	PreparacionTests test_7_no_se_puede_finalizar_con_faltantes	No se finaliza una preparación incompleta (15 de 20) y el estado no avanza.	**Aprobada**
PA-13	Despacho	DespachoTests test_d2_d4_d5_d6_preparacion_completa_genera_despacho	El despacho queda Entregado, la reserva pasa a Despachado y lo enviado coincide con lo preparado.	**Aprobada**

PA-14	Despacho	DespachoTests test_d1_preparacion_incompleta_no_se_despacha test_d7_despacho_no_descuenta_stock_dos_veces	No se despacha sin preparación finalizada y el despacho no vuelve a descontar stock.	**Aprobada**
PA-15	Devolución	DevolucionTests test_devolucion_completa_libera_stock_y_permite_cerrar	La devolución completa libera el stock (disponible vuelve a 100), pasa a Finalizado («Devuelto») y permite cerrar.	**Aprobada**
PA-16	Devolución	DevolucionTests test_ejemplo_80_buenas_5_danadas_2_perdidas_13_pendientes	80 buenas, 5 dañadas, 2 perdidas y 13 pendientes: stock 198/193, mermas Dañado 5 y Perdido 2, 13 siguen comprometidas.	**Aprobada**
PA-17	Merma	MermaTests test_merma_definitiva_baja_total_y_trabajable_una_sola_vez test_merma_no_se_borra_se_anula_y_restituye	La merma baja el stock (100 → 97) con su movimiento; no se borra: se anula y el stock se restituye.	**Aprobada**
PA-18	Merma	ValorMermaTests test_merma_sin_valor_se_valoriza_con_el_costo_de_reposicion test_sin_costo_usa_el_precio_del_producto	Valor = cantidad × costo de reposición (2 × $2.500 = $5.000); sin costo cargado se usa el precio (3 × $500 = $1.500).	**Aprobada**
PA-19	Historial	FlujoCompletoTests test_caso_integral	Flujo cotizador → confirmación → preparación → despacho con historial en el orden real de las acciones.	**Aprobada**
PA-20	Seguridad	SeguridadTests test_todas_las_secciones_del_panel_exigen_sesion_y_rol	Las 13 secciones del panel redirigen sin sesión (302) y rechazan a un rol sin permisos (403).	**Aprobada**
PA-21	Seguridad	CuentaClienteTests test_login_de_cliente_y_sin_acceso_al_panel test_login_no_redirige_a_sitios_externos	Un cliente no accede al panel, una contraseña errónea se rechaza y el login no redirige a sitios externos.	**Aprobada**
PA-22	Seguridad	SeguridadTests test_contrasena_antigua_en_texto_plano_se_cifra_al_entrar	Las contraseñas quedan cifradas con pbkdf2.	**Aprobada**
PA-23	Concurrencia	ConcurrenciaTests test_confirmaciones_simultaneas	Con dos conexiones simultáneas solo una confirmación puede pasar (requiere PostgreSQL).	Omitida
PA-24	Categorías	GestionCategoriasTests test_admin_crea_categoria_y_aparece_en_formulario_de_producto test_nombre_vacio_se_rechaza	El administrador crea una categoría con el nombre sin espacios sobrantes; un nombre vacío se rechaza.	**Aprobada**
PA-25	Categorías	GestionCategoriasTests test_nombre_duplicado_ignora_mayusculas_y_espacios test_restriccion_de_base_impide_duplicado_por_mayusculas	No se aceptan nombres repetidos (sin distinguir mayúsculas ni espacios) en el mismo negocio, y la base lo respalda con una restricción única.	**Aprobada**
PA-26	Categorías	GestionCategoriasTests test_no_se_elimina_categoria_con_productos test_no_se_puede_modificar_categoria_de_otro_negocio	Una categoría con productos no se elimina y una categoría de otro negocio no se puede modificar.	**Aprobada**

PA-27	Categorías	GestionCategoriasTests test_cambiar_categoria_del_producto_no_altera_stock_ni_precio test_producto_nuevo_no_acepta_categoria_inactiva_ni_ajena	Cambiar la categoría de un producto no altera stock ni precio; un producto nuevo no acepta categorías inactivas ni ajenas.	**Aprobada**
PA-28	Categorías	GestionCategoriasTests test_desactivar_oculta_categoria_del_catalogo_y_conserva_productos test_desactivar_no_altera_reservas_ni_detalles_historicos	Al desactivar una categoría deja de verse en el catálogo, pero se conservan sus productos, reservas y detalles.	**Aprobada**
PA-29	Seguridad	GestionCategoriasTests test_sin_sesion_no_crea_categorias test_usuario_que_no_es_administrador_recibe_403	Sin sesión se redirige al inicio de sesión y un usuario sin rol de administrador recibe rechazo (403).	**Aprobada**
PA-30	Catálogo	GestionCategoriasTests test_producto_sin_foto_usa_imagen_predeterminada	Un producto sin fotografía muestra una imagen predeterminada en el catálogo y en su ficha, nunca una imagen rota.	**Aprobada**
PA-31	Clientes	ValidacionesTests test_normaliza_celular_chileno	Los celulares chilenos se normalizan a +56 9 1234 5678 sin importar si se escriben con +56, 56, 0056, 09 o solo el número.	**Aprobada**
PA-32	Clientes	TelefonosGuardadosTests test_migracion_ordena_los_telefonos_ya_guardados	La migración 0010 ordena los teléfonos ya guardados y no altera los números que no son celulares chilenos.	**Aprobada**

PA-23 se omite automáticamente en SQLite: necesita dos conexiones reales a PostgreSQL para probar el bloqueo de filas. La regla de no sobreventa queda cubierta en forma secuencial por PA-08.

4. Pruebas representativas

Para cada grupo se muestra el código real de la prueba (con sus números de línea), los datos usados, el resultado esperado y el obtenido.

Clientes y validaciones (PA-01, PA-02)

Objetivo	Verificar que el RUT se valide y normalice, y que una solicitud del cotizador cree el cliente y el servicio correctos.
Datos utilizados	RUT 12.345.678-k y 12.345.678-5; solicitud de Ana María Pérez para 80 invitados con categorías Vajilla y Cristalería.
Resultado esperado	RUT normalizado, dígito verificador validado y servicio en estado Cotización sin detalles de artículos.
Resultado obtenido	Todas las afirmaciones se cumplieron.
Estado	APROBADA

Inventario/tests.py · Clientes: validaciones y creación desde el cotizador — Código real del repositorio desplegado

```
75  class ValidacionesTests(TestCase):
… líneas 76–78
76      def test_normaliza_rut(self):
77          self.assertEqual(normalizar_rut("12.345.678-k"), "12345678K")
78          self.assertEqual(normalizar_rut(" 12345678-5 "), "123456785")
… líneas 80–83
80      def test_digito_verificador(self):
81          self.assertTrue(rut_valido("123456785"))
82          self.assertFalse(rut_valido("123456789"))
83          self.assertFalse(rut_valido("12A"))
… class CotizadorPublicoTests(TestCase) · líneas 123–142
123     def test_solicitud_valida_crea_cliente_y_servicio(self):
124         respuesta = self.enviar()
125         self.assertEqual(respuesta.status_code, 200)
126         cuerpo = respuesta.json()
127         self.assertTrue(cuerpo["ok"])
128         self.assertEqual(set(cuerpo), {"ok", "message"})
129
130         cliente = Cliente.objects.get()
131         self.assertEqual(cliente.rut, "123456785")
132         self.assertEqual((cliente.nombre, cliente.apellido), ("Ana", "María Pérez"))
133
134         servicio = Servicio.objects.get()
135         self.assertEqual(servicio.estado, Servicio.Estado.COTIZACION)
136         self.assertEqual(servicio.cantidad_personas, 80)
137         self.assertTrue(servicio.requiere_despacho)
138         self.assertEqual(servicio.medio_contacto, "WhatsApp")
139         self.assertEqual(servicio.comentarios, "Matrimonio al aire libre.")
140         self.assertEqual(set(servicio.categorias_solicitadas.all()), {self.vajilla, self.cristaleria})
141         # Solo se guarda lo que pidió el cliente: sin artículos ni cantidades inventadas.
142         self.assertFalse(DetalleServicio.objects.exists())
```

Figura 2. Código de las pruebas: clientes y validaciones (02_Codigo_Pruebas_Validaciones.png)

Cotización (PA-04 y confirmación sin stock)

Objetivo	Verificar la asignación de productos, el cálculo del total y que no se confirme una reserva sin stock.
Datos utilizados	Plato principal a $500 y copa a $300; 45 platos y 20 copas. Otra reserva de 80 platos el mismo día.
Resultado esperado	Total = 45 × 500 + 20 × 300 = $28.500. Al confirmar 40 platos con solo 20 libres, el estado sigue en Cotización y se informa el faltante.
Resultado obtenido	La prueba finalizó correctamente y las afirmaciones fueron aprobadas.
Estado	APROBADA

Inventario/tests.py · Cotización: asignación de productos, total y confirmación sin stock — Código real del repositorio desplegado

… class AdministracionTests(TestCase) · líneas 283–303

```
283      def test_catalogo_agrega_varios_productos_de_una_vez(self):
284          self.iniciar_sesion()
285          copa = Articulo.objects.create(negocio=self.negocio, categoria=self.vajilla, codigo="vaj-009", nombre="Copa",
286                                         precio_arriendo=300, stock_total=50, stock_trabajable=50)
287          url = reverse("gestion_cotizacion", args=[self.servicio.id])
288          catalogo = self.client.get(url).context["catalogo"]
289          # Cada producto trae la disponibilidad para la fecha del evento.
290          self.assertEqual(catalogo[0]["etiqueta"], "Vajilla")
291          self.assertEqual({a.nombre: a.disponible_fecha for a in catalogo[0]["productos"]}, {"Copa": 50, "Plato principal": 100})
292
293          respuesta = self.client.post(url, {
294              "accion": "agregar_articulos",
295              "articulo_id": [self.plato.id, copa.id],
296              "cantidad": ["40", ""],
297          }, follow=True)
298          self.assertContains(respuesta, "Se agregaron 1 producto: 40 × Plato principal")
299          self.client.post(url, {"accion": "agregar_articulos", "articulo_id": [self.plato.id, copa.id], "cantidad": ["5", "20"]})
300          cantidades = dict(self.servicio.detalles.values_list("articulo__nombre", "cantidad"))
301          self.assertEqual(cantidades, {"Plato principal": 45, "Copa": 20})
302          self.servicio.refresh_from_db()
303          self.assertEqual(self.servicio.total, 45 * 500 + 20 * 300)
```

… líneas 328–336

```
328      def test_confirmar_sin_stock_no_cambia_el_estado(self):
329          self.iniciar_sesion()
330          crear_servicio(self.negocio, self.plato, 80, EN_10_DIAS)
331          url = reverse("gestion_cotizacion", args=[self.servicio.id])
332          self.client.post(url, {"accion": "agregar_articulo", "articulo_id": self.plato.id, "cantidad": 40})
333          respuesta = self.client.post(url, {"accion": "confirmar"}, follow=True)
334          self.servicio.refresh_from_db()
335          self.assertEqual(self.servicio.estado, Servicio.Estado.COTIZACION)
336          self.assertContains(respuesta, "Plato principal: solicitado 40, disponible 20")
```

Figura 3. Código de las pruebas: cotización (03_Codigo_Pruebas_Cotizacion.png)

Reserva y disponibilidad (PA-07, PA-08)

Objetivo	Verificar que la confirmación comprometa unidades y que el sistema impida reservar más stock del disponible.
Datos utilizados	Artículo con 100 unidades; reservas de 30 + 70, de 70 + 40 y dos de 60.
Resultado esperado	Comprometido 100 y disponible 0; la confirmación que excede se rechaza con StockInsuficiente (40 / 30 / 10) y la segunda de 60 no pasa.
Resultado obtenido	Aprobadas.
Estado	APROBADA

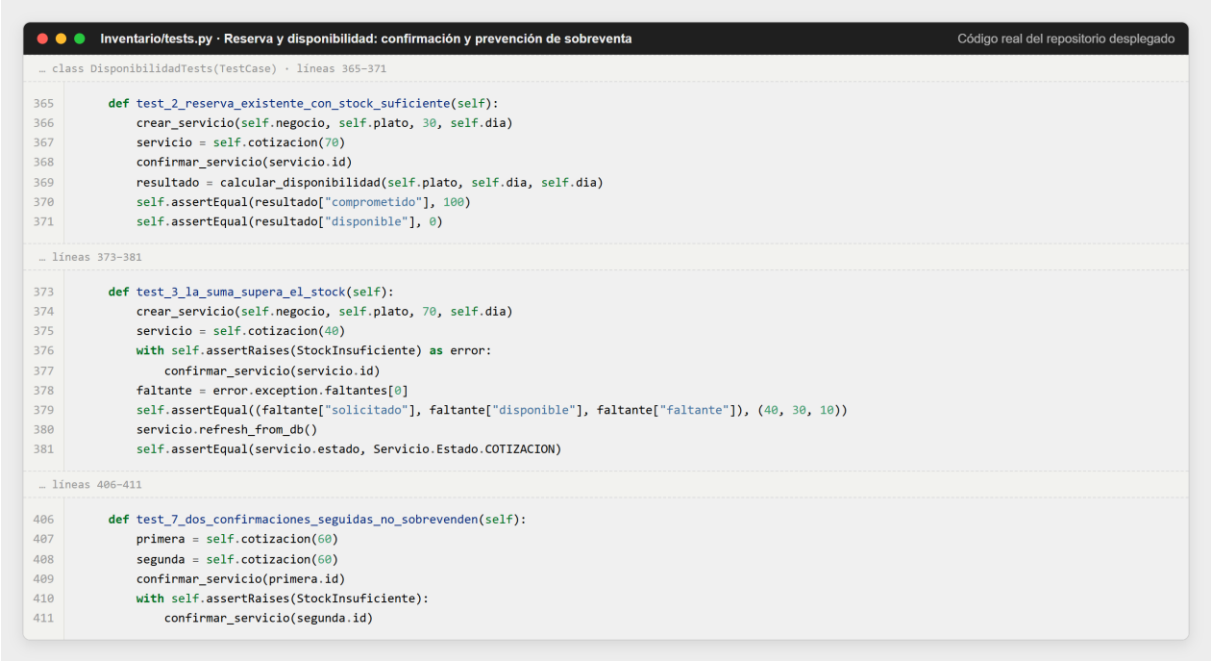


Figura 4. Código de las pruebas: reserva y disponibilidad (04_Codigo_Pruebas_Disponibilidad.png)

Preparación y despacho (PA-11, PA-13)

Objetivo	Verificar los cambios de estado y el historial al preparar, y que el despacho registre exactamente lo preparado.
Datos utilizados	Reserva confirmada de 40 platos y 20 copas.
Resultado esperado	Confirmado → Preparacion → Preparado con tres registros de historial; despacho Entregado, reserva Despachado y detalles iguales a lo preparado.
Resultado obtenido	Aprobadas.
Estado	APROBADA

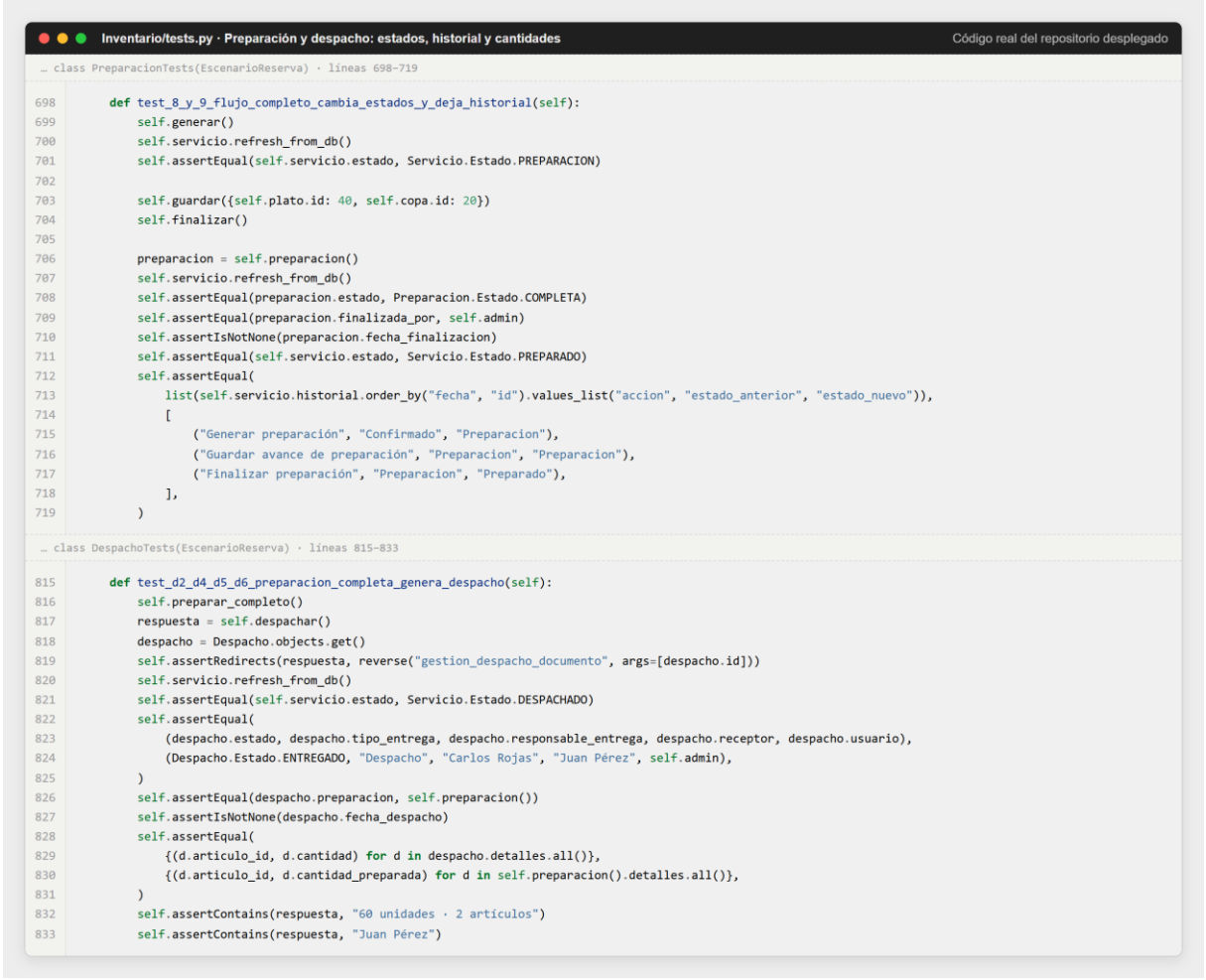


Figura 5. Código de las pruebas: preparación y despacho (05_Codigo_Pruebas_Preparacion_Despacho.png)

Devolución y merma (PA-16, PA-18)

Objetivo	Verificar la diferenciación entre buenas, dañadas, perdidas y pendientes, y la valorización de las mermas.
Datos utilizados	100 platos despachados: 80 buenos, 5 dañados, 2 perdidos y 13 pendientes. Mermas de 2 y 3 unidades.
Resultado esperado	Stock 198 total / 193 trabajable; mermas Dañado 5 y Perdido 2; 13 siguen comprometidas. Valor 2 × $2.500 = $5.000 y, sin costo, 3 × $500 = $1.500.
Resultado obtenido	Aprobadas.
Estado	APROBADA

Inventario/tests.py · Devolución y merma: buenas, dañadas, perdidas, pendientes y valorización — Código real del repositorio desplegado

… class DevolucionTests(EscenarioReserva) · líneas 1052–1078

```
1052      def test_ejemplo_80_buenas_5_danadas_2_perdidas_13_pendientes(self):
1053          self.plato.stock_total = self.plato.stock_trabajable = 200
1054          self.plato.save()
1055          self.servicio.detalles.filter(articulo=self.plato).update(cantidad=100)
1056          self.despachar_todo_con(100)
1057          self.devolver({self.plato: (80, 5, 2), self.copa: (20, 0, 0)})
1058
1059          self.servicio.refresh_from_db()
1060          self.plato.refresh_from_db()
1061          self.assertEqual(self.servicio.estado, Servicio.Estado.DEVOLUCION_PARCIAL)
1062          # 5 dañadas bajan solo el trabajable; 2 perdidas bajan total y trabajable.
1063          self.assertEqual((self.plato.stock_total, self.plato.stock_trabajable), (198, 193))
1064          # Siguen comprometidas las 13 pendientes: 193 - 13 = 180 disponibles.
1065          self.assertEqual(self.disponible(self.plato), 180)
1066          self.assertEqual(
1067              sorted(Merma.objects.filter(articulo=self.plato).values_list("tipo", "cantidad")),
1068              [("Dañado", 5), ("Perdido", 2)],
1069          )
1070          fila = next(a for a in self.client.get(reverse("gestion_inventario")).context["inventario"] if a.id == self.plato.id)
1071          self.assertEqual(fila.fuera_de_bodega, 13)
1072
1073          # Vuelven las 13: se liberan, queda Devuelto y se puede cerrar.
1074          self.devolver({self.plato: (13, 0, 0)})
1075          self.servicio.refresh_from_db()
1076          self.assertEqual(self.servicio.estado, Servicio.Estado.FINALIZADO)
1077          self.assertEqual(self.disponible(self.plato), 193)
1078          self.assertEqual(Devolucion.objects.count(), 2)
```

… class ValorMermaTests(EscenarioReserva) · líneas 1405–1410

```
1405      def test_merma_sin_valor_se_valoriza_con_el_costo_de_reposicion(self):
1406          Articulo.objects.filter(id=self.plato.id).update(costo_reposicion=2500)
1407          self.client.post(reverse("gestion_mermas"), {"accion": "crear", "articulo_id": self.plato.id, "tipo": "Roto",
1408                                                       "cantidad": "2", "motivo": "Caída", "valor_reposicion": ""})
1409          self.assertEqual(Merma.objects.get().valor_reposicion, 5000)
1410          self.assertEqual(self.client.get(reverse("gestion_mermas")).context["kpis"]["valor"], 5000)
```

… líneas 1412–1416

```
1412      def test_sin_costo_usa_el_precio_del_producto(self):
1413          self.client.post(reverse("gestion_mermas"), {"accion": "crear", "articulo_id": self.plato.id, "tipo": "Roto",
1414                                                       "cantidad": "3", "motivo": "Caída", "valor_reposicion": ""})
1415          self.assertEqual(Merma.objects.get().valor_reposicion, 1500)  # 3 × precio $500
1416          self.assertEqual(self.client.get(reverse("gestion_mermas")).context["kpis"]["sin_costo"], 0)
```

Figura 6. Código de las pruebas: devolución y merma (06_Codigo_Pruebas_Devolucion_Merma.png)

Seguridad y permisos (PA-20)

Objetivo	Verificar que ninguna sección del panel sea accesible sin sesión o sin el rol adecuado, tanto para ver como para modificar.
Datos utilizados	13 secciones del panel; peticiones GET y POST sin sesión y con un rol «Invitado».
Resultado esperado	Sin sesión: redirección al login (302). Con rol sin permisos: rechazo (403). Ninguna petición modifica datos.
Resultado obtenido	Aprobada.
Estado	APROBADA

Inventario/tests.py · Seguridad: acceso sin sesión y sin permisos — Código real del repositorio desplegado

… class SeguridadTests(EscenarioReserva) · líneas 1457–1473

```
1457      def test_todas_las_secciones_del_panel_exigen_sesion_y_rol(self):
1458          nombres = ["inicio", "gestion_clientes", "gestion_cotizaciones", "gestion_reservas", "gestion_preparacion",
1459                     "gestion_despachos", "gestion_devoluciones", "gestion_mermas", "gestion_inventario",
1460                     "gestion_calendario", "gestion_reportes", "gestion_usuarios", "gestion_configuracion"]
1461          # El modelo exige rol: "sin permisos" se prueba con un rol distinto de Administrador.
1462          bodega = Usuario.objects.create(negocio=self.negocio, rol=Rol.objects.create(nombre="Invitado"), nombre="S", apellido="R",
1463                                          email="s@ejemplo.cl", password="x")
1464          self.client.logout()
1465          for nombre in nombres:
1466              with self.subTest(vista=nombre, caso="sin sesión"):
1467                  self.assertEqual(self.client.get(reverse(nombre)).status_code, 302)
1468                  self.assertEqual(self.client.post(reverse(nombre), {"accion": "crear"}).status_code, 302)
1469          self.iniciar_sesion(bodega)
1470          for nombre in nombres:
1471              with self.subTest(vista=nombre, caso="sin permisos"):
1472                  self.assertEqual(self.client.get(reverse(nombre)).status_code, 403)
1473                  self.assertEqual(self.client.post(reverse(nombre), {"accion": "crear"}).status_code, 403)
```

Figura 7. Código de las pruebas: seguridad y permisos (07_Codigo_Pruebas_Seguridad.png)

Gestión de categorías (PA-24 a PA-29)

Objetivo	Verificar que el administrador pueda crear, renombrar, desactivar y eliminar categorías con las reglas del negocio (nombre único, sin eliminar categorías con productos, sin afectar stock, precios ni reservas).
Datos utilizados	Categorías «Iluminación», «ILUMINACIÓN» y « iluminación »; categoría con un producto de 100 unidades; reserva confirmada de 20 unidades; categoría de otro negocio.
Resultado esperado	Solo se crea una categoría «Iluminación»; la categoría con productos no se elimina y muestra el mensaje de bloqueo; al cambiar de categoría el producto conserva stock y precio; al desactivarla desaparece del catálogo y la reserva sigue confirmada.
Resultado obtenido	Aprobadas.
Estado	APROBADA

Inventario/tests.py · Gestión de categorías: nombre único, eliminación, cambio de categoría y catálogo — Código real del repositorio desplegado

```
… class GestionCategoriasTests(TestCase) · líneas 1879–1887

1879      def test_nombre_duplicado_ignora_mayusculas_y_espacios(self):
1880          self.iniciar_sesion()
1881          self.crear("Iluminación")
1882          for variante in ("iluminación", " ILUMINACIÓN ", "Iluminación"):
1883              self.crear(variante)
1884          self.assertEqual(
1885              [c.nombre for c in Categoria.objects.filter(negocio=self.negocio, nombre__icontains="lumin")],
1886              ["Iluminación"],
1887          )

… líneas 1931–1935

1931      def test_no_se_elimina_categoria_con_productos(self):
1932          self.iniciar_sesion()
1933          respuesta = self.client.post(self.url, {"accion": "eliminar", "categoria_id": self.vajilla.id}, follow=True)
1934          self.assertTrue(Categoria.objects.filter(id=self.vajilla.id).exists())
1935          self.assertContains(respuesta, "No se puede eliminar la categoría porque contiene productos.")

… líneas 1970–1981

1970      def test_cambiar_categoria_del_producto_no_altera_stock_ni_precio(self):
1971          self.iniciar_sesion()
1972          antes = (self.plato.stock_total, self.plato.stock_trabajable, self.plato.precio_arriendo)
1973          self.client.post(reverse("gestion_producto", args=[self.plato.id]), {
1974              "accion": "editar", "nombre": self.plato.nombre, "categoria_id": self.cristaleria.id,
1975              "precio_arriendo": self.plato.precio_arriendo, "costo_reposicion": self.plato.costo_reposicion,
1976              "stock_total": self.plato.stock_total, "stock_trabajable": self.plato.stock_trabajable, "activo": "on",
1977          })
1978          self.plato.refresh_from_db()
1979          self.assertEqual(self.plato.categoria, self.cristaleria)
1980          self.assertEqual((self.plato.stock_total, self.plato.stock_trabajable, self.plato.precio_arriendo), antes)
1981          self.assertFalse(MovimientoInventario.objects.filter(articulo=self.plato).exists())

… líneas 1991–2003

1991      def test_desactivar_oculta_categoria_del_catalogo_y_conserva_productos(self):
1992          self.iniciar_sesion()
1993          self.client.post(self.url, {"accion": "desactivar", "categoria_id": self.vajilla.id})
1994          self.vajilla.refresh_from_db()
1995          self.assertFalse(self.vajilla.activo)
1996          self.assertTrue(Articulo.objects.filter(id=self.plato.id).exists())
1997          catalogo = self.client.get(reverse("catalogo"))
1998          self.assertNotIn(self.vajilla, catalogo.context["categorias"])
1999          self.assertNotIn(self.plato, catalogo.context["productos"])
2000          self.client.post(self.url, {"accion": "activar", "categoria_id": self.vajilla.id})
2001          catalogo = self.client.get(reverse("catalogo"))
2002          self.assertIn(self.vajilla, catalogo.context["categorias"])
2003          self.assertIn(self.plato, catalogo.context["productos"])
```

Figura 8. Código de las pruebas: gestión de categorías (12_Codigo_Pruebas_Categorias.png)

5. Ejecución de la suite

Se ejecutó la suite completa con «python manage.py test --settings=Epyc.settings_test --verbosity 2». Django creó la base de datos de prueba temporal en memoria (memorydb_default), aplicó las 10 migraciones del proyecto, corrió las 160 pruebas en 47,801 segundos y eliminó la base al terminar, como se ve en la primera y la última línea de la salida.

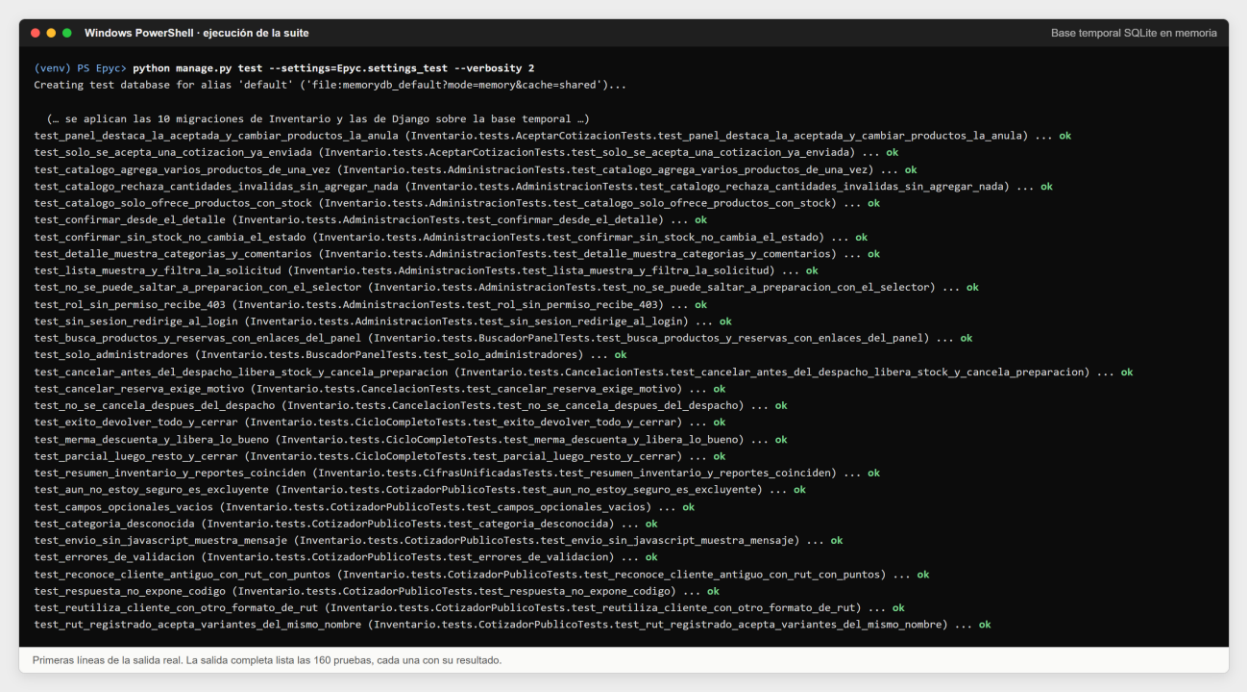


Figura 9. Inicio de la ejecución: comando, base temporal y primeras pruebas (08_Ejecucion_Pruebas.png)

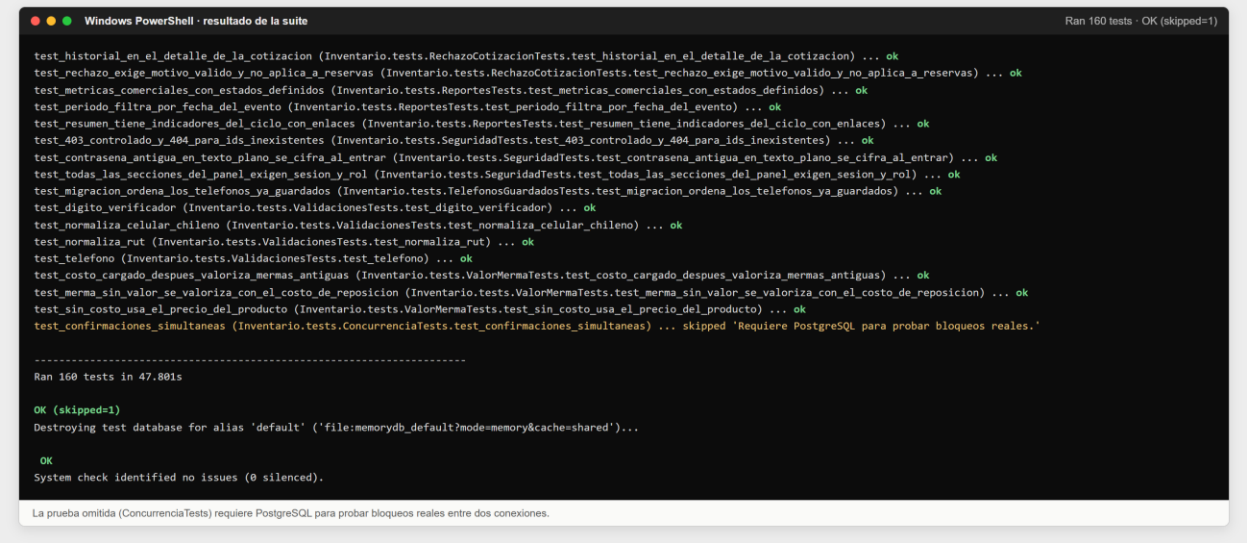


Figura 10. Fin de la ejecución: Ran 160 tests · OK (skipped=1) (09_Resultados_Pruebas_OK.png)

6. Cobertura

La cobertura mide qué porcentaje de las instrucciones del código de la aplicación se ejecutó durante las pruebas. Se excluyen las migraciones, el propio archivo de pruebas y las plantillas.

Módulo	Instrucciones	Sin cubrir	Cobertura
Inventario/validaciones.py	38	0	**100 %**
Inventario/disponibilidad.py	97	3	**97 %**
Inventario/models.py	366	18	**95 %**
Inventario/operaciones.py	394	24	**94 %**
Inventario/views.py	1.628	319	80 %
Otros (urls, apps, admin, gestion)	13	1	92 %
Total	**2.536**	**365**	**86 %**

Interpretación

- **Reglas de negocio:** la lógica de disponibilidad, operaciones (preparación, despacho, devolución, merma) y modelos supera el 94 %, y las validaciones llegan al 100 %.
- **Vistas:** 80 %. Lo no cubierto corresponde principalmente a ramas de presentación y mensajes de error poco frecuentes (por ejemplo, formularios incompletos o filtros combinados del panel).
- **gestion/views.py:** 0 %, pero es un archivo de una sola línea sin uso; la aplicación funciona desde Inventario.
- **Limitación:** la prueba de concurrencia (PA-23) no corre en SQLite; las líneas de bloqueo simultáneo solo se ejercitan en PostgreSQL.

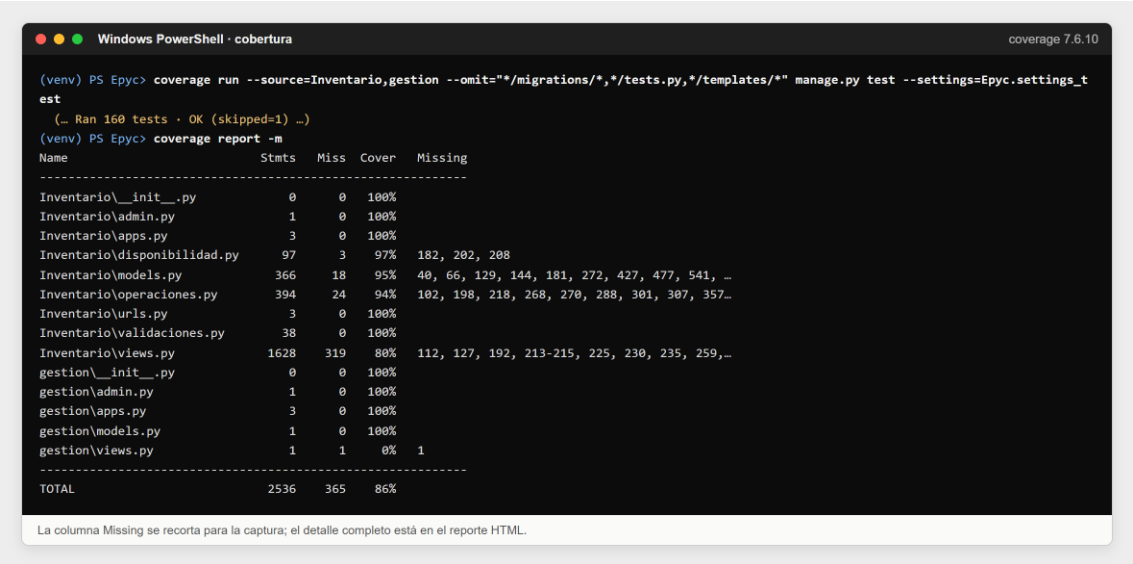
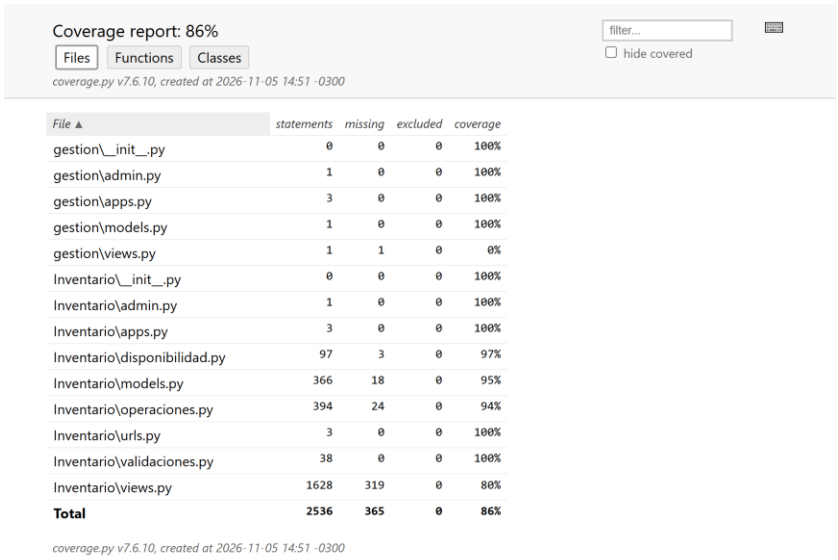


Figura 11. Reporte de cobertura en consola (coverage report -m) (10_Cobertura_Reporte.png)



Coverage report: 86%

Files | Functions | Classes

filter...

hide covered

coverage.py v7.6.10, created at 2026-11-05 14:51 -0300

File ▲	statements	missing	excluded	coverage
gestion__init__.py	0	0	0	100%
gestion\admin.py	1	0	0	100%
gestion\apps.py	3	0	0	100%
gestion\models.py	1	0	0	100%
gestion\views.py	1	1	0	0%
Inventario__init__.py	0	0	0	100%
Inventario\admin.py	1	0	0	100%
Inventario\apps.py	3	0	0	100%
Inventario\disponibilidad.py	97	3	0	97%
Inventario\models.py	366	18	0	95%
Inventario\operaciones.py	394	24	0	94%
Inventario\urls.py	3	0	0	100%
Inventario\validaciones.py	38	0	0	100%
Inventario\views.py	1628	319	0	80%
Total	**2536**	**365**	**0**	**86%**

coverage.py v7.6.10, created at 2026-11-05 14:51 -0300

Figura 12. Reporte de cobertura HTML (coverage html) (11_Cobertura_HTML.png)

7. Correcciones previas a la ejecución

Al preparar esta evidencia se detectaron dos problemas en la configuración de pruebas del repositorio. No afectaban a la aplicación publicada, pero impedían una ejecución limpia. Se corrigieron y luego se ejecutó la suite completa nuevamente.

Commit	Problema	Corrección
9720890	«python manage.py test» fallaba al buscar pruebas: gestion/templates/__init__.py contenía HTML y Python lo intentaba importar.	Se eliminó ese archivo, una copia antigua de una plantilla que no se usaba.
43ad735	Con settings_test.py fallaban 2 pruebas de seguridad: usaba cifrado MD5 y las pruebas verifican el cifrado real pbkdf2. Además, la consola mostraba los rechazos 400/403 que las pruebas provocan a propósito.	Se usa el mismo cifrado que producción y se silencian esos registros solo durante las pruebas.

8. Resguardo de la información

Esta evidencia no incluye ni sube:

- La base de datos real ni archivos .sqlite3: las pruebas usan una base temporal en memoria.
- Archivos .env, contraseñas, tokens de Supabase, DATABASE_URL ni claves de Render.
- Cookies, sesiones ni datos personales reales: las pruebas crean datos ficticios.
- La carpeta del entorno virtual ni el archivo .coverage.

Conclusión

La ejecución de las pruebas automáticas permitió verificar el correcto funcionamiento de las principales reglas de negocio del sistema. Las pruebas comprobaron la creación de cotizaciones, los cálculos monetarios, los cambios de estado, el control del inventario, el registro de devoluciones y mermas, y las restricciones de acceso. La suite de 160 pruebas terminó con 159 aprobadas, 1 omitida por requerir PostgreSQL y ninguna fallida, con una cobertura del 86 %. Se ejecutó en un entorno de pruebas independiente, sin modificar los datos de producción.

Anexo. Lista completa de pruebas ejecutadas

Resultado de cada prueba según la salida de «python manage.py test --settings=Epyc.settings_test --verbosity 2».

ValidacionesTests · Validación de RUT y teléfono (4)

Prueba	Resultado
test_digito_verificador	**Aprobada**
test_normaliza_celular_chileno	**Aprobada**
test_normaliza_rut	**Aprobada**
test_telefono	**Aprobada**

CotizadorPublicoTests · Cotizador público (solicitud del cliente) (11)

Prueba	Resultado
test_aun_no_estoy_seguro_es_excluyente	**Aprobada**
test_campos_opcionales_vacios	**Aprobada**
test_categoria_desconocida	**Aprobada**
test_envio_sin_javascript_muestra_mensaje	**Aprobada**
test_errores_de_validacion	**Aprobada**
test_reconoce_cliente_antiguo_con_rut_con_puntos	**Aprobada**
test_respuesta_no_expone_codigo	**Aprobada**
test_reutiliza_cliente_con_otro_formato_de_rut	**Aprobada**
test_rut_registrado_acepta_variantes_del_mismo_nombre	**Aprobada**
test_rut_registrado_con_otro_nombre_se_rechaza	**Aprobada**
test_solicitud_valida_crea_cliente_y_servicio	**Aprobada**

AdministracionTests · Gestión de cotizaciones en el panel (10)

Prueba	Resultado
test_catalogo_agrega_varios_productos_de_una_vez	**Aprobada**
test_catalogo_rechaza_cantidades_invalidas_sin_agregar_nada	**Aprobada**
test_catalogo_solo_ofrece_productos_con_stock	**Aprobada**
test_confirmar_desde_el_detalle	**Aprobada**
test_confirmar_sin_stock_no_cambia_el_estado	**Aprobada**
test_detalle_muestra_categorias_y_comentarios	**Aprobada**
test_lista_muestra_y_filtra_la_solicitud	**Aprobada**
test_no_se_puede_saltar_a_preparacion_con_el_selector	**Aprobada**

test_rol_sin_permiso_recibe_403	**Aprobada**
test_sin_sesion_redirige_al_login	**Aprobada**

DisponibilidadTests · Disponibilidad y confirmación de reservas (8)

Prueba	Resultado
test_1_sin_reservas_superpuestas	**Aprobada**
test_2_reserva_existente_con_stock_suficiente	**Aprobada**
test_3_la_suma_supera_el_stock	**Aprobada**
test_4_reserva_en_otras_fechas_no_afecta	**Aprobada**
test_5_cotizacion_no_confirmada_no_bloquea	**Aprobada**
test_6_el_mismo_servicio_no_se_cuenta_dos_veces	**Aprobada**
test_7_dos_confirmaciones_seguidas_no_sobrevenden	**Aprobada**
test_rangos_que_se_tocan_si_se_superponen	**Aprobada**

InventarioPanelTests · Inventario en el panel (7)

Prueba	Resultado
test_crear_producto_con_codigo_automatico	**Aprobada**
test_editar_stock_vuelve_al_listado_y_registra_movimiento	**Aprobada**
test_ficha_es_la_del_panel	**Aprobada**
test_inventario_agrupado_por_categoria_con_totales	**Aprobada**
test_listado_muestra_comprometido_y_disponible	**Aprobada**
test_resumen_y_reportes_cuentan_solo_reservas	**Aprobada**
test_stock_invalido_no_se_guarda	**Aprobada**

ConcurrenciaTests · Confirmaciones simultáneas (PostgreSQL) (1)

Prueba	Resultado
test_confirmaciones_simultaneas	Omitida

PreparacionTests · Preparación de pedidos (18)

Prueba	Resultado
test_10_usuario_sin_permisos_recibe_rechazo	**Aprobada**
test_1_reserva_confirmada_genera_preparacion_con_sus_articulos	**Aprobada**
test_2_cotizacion_sin_confirmar_no_genera_preparacion	**Aprobada**
test_3_no_se_crean_dos_preparaciones	**Aprobada**

test_5_y_6_cantidades_invalidas_se_rechazan_sin_guardar_nada	**Aprobada**
test_7_no_se_puede_finalizar_con_faltantes	**Aprobada**
test_8_y_9_flujo_completo_cambia_estados_y_deja_historial	**Aprobada**
test_articulo_ajeno_o_repetido_se_rechaza	**Aprobada**
test_avance_sin_observacion_por_articulo_conserva_la_existente	**Aprobada**
test_guardar_avance_registra_cantidades_y_observacion	**Aprobada**
test_hoja_lista_por_categoria_y_una_hoja_impresa_por_categoria	**Aprobada**
test_listado_muestra_pendientes_progreso_y_filtros	**Aprobada**
test_marcar_pedido_como_preparado_habilita_el_despacho	**Aprobada**
test_preparacion_completa_no_se_puede_editar	**Aprobada**
test_preparado_sigue_reteniendo_stock_y_no_permite_editar_articulos	**Aprobada**
test_reserva_cancelada_no_admite_avance	**Aprobada**
test_reserva_sin_articulos_o_con_evento_pasado_no_genera	**Aprobada**
test_selector_de_estado_solo_ofrece_transiciones_permitidas	**Aprobada**

DespachoTests · Despacho (13)

Prueba	Resultado
test_d10_usuario_sin_permisos_no_puede_despachar	**Aprobada**
test_d1_preparacion_incompleta_no_se_despacha	**Aprobada**
test_d2_d4_d5_d6_preparacion_completa_genera_despacho	**Aprobada**
test_d3_no_se_despacha_dos_veces	**Aprobada**
test_d7_despacho_no_descuenta_stock_dos_veces	**Aprobada**
test_d8_aparece_en_calendario_resumen_y_listado	**Aprobada**
test_d9_si_algo_falla_no_queda_nada_a_medias	**Aprobada**
test_datos_invalidos_se_rechazan_y_se_conservan	**Aprobada**
test_fecha_de_devolucion_define_el_termino_de_la_reserva	**Aprobada**
test_fecha_de_devolucion_invalida_o_sin_stock	**Aprobada**
test_listado_separa_pendientes_y_filtra	**Aprobada**
test_no_se_envia_mas_de_lo_preparado_ni_nada	**Aprobada**
test_se_puede_enviar_menos_de_lo_preparado	**Aprobada**

FlujoCompletoTests · Flujo integral cotización → despacho (1)

Prueba	Resultado
test_caso_integral	**Aprobada**

DevolucionTests · Devoluciones (14)

Prueba	Resultado
test_calendario_muestra_devolucion_esperada_y_luego_recibida	**Aprobada**
test_detalle_del_cliente_muestra_el_ciclo	**Aprobada**
test_devolucion_completa_libera_stock_y_permite_cerrar	**Aprobada**
test_ejemplo_80_buenas_5_danadas_2_perdidas_13_pendientes	**Aprobada**
test_error_se_muestra_en_la_fila_y_conserva_los_datos	**Aprobada**
test_formulario_con_columna_unica_de_merma	**Aprobada**
test_formulario_exige_definir_todas_las_unidades	**Aprobada**
test_movimientos_explican_el_ciclo	**Aprobada**
test_no_se_cierra_con_pendientes	**Aprobada**
test_no_se_devuelve_dos_veces_la_misma_unidad_ni_articulos_ajenos	**Aprobada**
test_no_se_devuelve_sin_despacho	**Aprobada**
test_pendientes_siguen_bloqueando_aunque_el_evento_haya_pasado	**Aprobada**
test_usuario_sin_permisos_no_registra_devoluciones_ni_mermas	**Aprobada**
test_validaciones_de_cantidades	**Aprobada**

MermaTests · Mermas (5)

Prueba	Resultado
test_dano_recuperable_baja_solo_trabajable_y_se_puede_reparar	**Aprobada**
test_filtros	**Aprobada**
test_merma_definitiva_baja_total_y_trabajable_una_sola_vez	**Aprobada**
test_merma_no_se_borra_se_anula_y_restituye	**Aprobada**
test_validaciones	**Aprobada**

CancelacionTests · Cancelación de reservas (3)

Prueba	Resultado
test_cancelar_antes_del_despacho_libera_stock_y_cancela_preparacion	**Aprobada**
test_cancelar_reserva_exige_motivo	**Aprobada**
test_no_se_cancela_despues_del_despacho	**Aprobada**

CicloCompletoTests · Ciclo completo con cierre (3)

Prueba	Resultado
test_exito_devolver_todo_y_cerrar	**Aprobada**

test_merma_descuenta_y_libera_lo_bueno	**Aprobada**
test_parcial_luego_resto_y_cerrar	**Aprobada**

ReportesTests · Reportes (3)

Prueba	Resultado
test_metricas_comerciales_con_estados_definidos	**Aprobada**
test_periodo_filtra_por_fecha_del_evento	**Aprobada**
test_resumen_tiene_indicadores_del_ciclo_con_enlaces	**Aprobada**

CifrasUnificadasTests · Cifras unificadas (resumen, inventario y reportes) (1)

Prueba	Resultado
test_resumen_inventario_y_reportes_coinciden	**Aprobada**

ValorMermaTests · Valorización de mermas (3)

Prueba	Resultado
test_costo_cargado_despues_valoriza_mermas_antiguas	**Aprobada**
test_merma_sin_valor_se_valoriza_con_el_costo_de_reposicion	**Aprobada**
test_sin_costo_usa_el_precio_del_producto	**Aprobada**

BuscadorPanelTests · Buscador del panel (2)

Prueba	Resultado
test_busca_productos_y_reservas_con_enlaces_del_panel	**Aprobada**
test_solo_administradores	**Aprobada**

SeguridadTests · Seguridad y permisos (3)

Prueba	Resultado
test_403_controlado_y_404_para_ids_inexistentes	**Aprobada**
test_contrasena_antigua_en_texto_plano_se_cifra_al_entrar	**Aprobada**
test_todas_las_secciones_del_panel_exigen_sesion_y_rol	**Aprobada**

CuentaClienteTests · Cuenta del cliente (11)

Prueba	Resultado

test_cambiar_contrasena	**Aprobada**
test_cotizar_con_cuenta_usa_sus_datos_y_ve_solo_lo_suyo	**Aprobada**
test_cuenta_exige_sesion	**Aprobada**
test_login_de_cliente_y_sin_acceso_al_panel	**Aprobada**
test_login_no_redirige_a_sitios_externos	**Aprobada**
test_no_se_duplican_cuentas	**Aprobada**
test_quien_ya_cotizo_activa_su_cuenta_solo_con_los_mismos_datos	**Aprobada**
test_registro_crea_cuenta_cifrada_e_inicia_sesion	**Aprobada**
test_reserva_confirmada_no_se_cancela_desde_la_cuenta	**Aprobada**
test_resumen_separa_en_curso_e_historial	**Aprobada**
test_validaciones_del_registro	**Aprobada**

AceptarCotizacionTests · Aceptación de cotización por el cliente (3)

Prueba	Resultado
test_aceptar_avisa_a_la_empresa_sin_confirmar_la_reserva	**Aprobada**
test_panel_destaca_la_aceptada_y_cambiar_productos_la_anula	**Aprobada**
test_solo_se_acepta_una_cotizacion_ya_enviada	**Aprobada**

FlujoCotizacionTests · Envío y confirmación de cotizaciones (5)

Prueba	Resultado
test_enviar_por_correo_avisa_y_pasa_a_contactado	**Aprobada**
test_marcar_enviada_por_whatsapp_y_luego_confirmar	**Aprobada**
test_no_se_envia_sin_productos_ni_por_get	**Aprobada**
test_pantalla_muestra_solo_el_siguiente_paso	**Aprobada**
test_si_falla_el_correo_avisa_y_no_cambia_el_estado	**Aprobada**

EnvioSegunPreferenciaTests · Envío según medio preferido (2)

Prueba	Resultado
test_boton_principal_segun_preferencia	**Aprobada**
test_si_falta_el_dato_usa_el_medio_disponible	**Aprobada**

RechazoCotizacionTests · Rechazo de cotizaciones (3)

Prueba	Resultado
test_cliente_rechaza_y_queda_cancelada_con_motivo	**Aprobada**

test_historial_en_el_detalle_de_la_cotizacion	**Aprobada**
test_rechazo_exige_motivo_valido_y_no_aplica_a_reservas	**Aprobada**

GestionCategoriasTests · Gestión de categorías (25)

Prueba	Resultado
test_admin_crea_categoria_y_aparece_en_formulario_de_producto	**Aprobada**
test_botones_de_categorias_estan_conectados_a_sus_modales	**Aprobada**
test_cambiar_categoria_del_producto_no_altera_stock_ni_precio	**Aprobada**
test_catalogo_oculta_categoria_activa_sin_productos_activos	**Aprobada**
test_categoria_nueva_con_producto_aparece_en_catalogo	**Aprobada**
test_cotizador_publico_solo_ofrece_categorias_activas	**Aprobada**
test_desactivar_no_altera_reservas_ni_detalles_historicos	**Aprobada**
test_desactivar_oculta_categoria_del_catalogo_y_conserva_productos	**Aprobada**
test_editar_nombre_y_rechazar_duplicado	**Aprobada**
test_editar_sin_cambiar_el_nombre_mostrado_conserva_el_codigo_interno	**Aprobada**
test_no_se_elimina_categoria_con_productos	**Aprobada**
test_no_se_puede_modificar_categoria_de_otro_negocio	**Aprobada**
test_nombre_duplicado_ignora_mayusculas_y_espacios	**Aprobada**
test_nombre_vacio_se_rechaza	**Aprobada**
test_otro_negocio_puede_repetir_el_nombre	**Aprobada**
test_panel_muestra_cantidad_de_productos_por_categoria	**Aprobada**
test_producto_existente_conserva_categoria_desactivada_en_su_formulario	**Aprobada**
test_producto_nuevo_en_categoria_nueva_conserva_stock_inicial	**Aprobada**
test_producto_nuevo_no_acepta_categoria_inactiva_ni_ajena	**Aprobada**
test_producto_sin_foto_usa_imagen_predeterminada	**Aprobada**
test_restriccion_de_base_impide_duplicado_por_mayusculas	**Aprobada**
test_se_elimina_categoria_vacia	**Aprobada**
test_selector_de_producto_nuevo_solo_muestra_categorias_activas	**Aprobada**
test_sin_sesion_no_crea_categorias	**Aprobada**
test_usuario_que_no_es_administrador_recibe_403	**Aprobada**

TelefonosGuardadosTests · Teléfonos guardados en formato uniforme (1)

Prueba	Resultado
test_migracion_ordena_los_telefonos_ya_guardados	**Aprobada**